

RAPPORT DE PROJET

API DE GESTION DES ÉTUDIANTS

Documentation, versions, tests et cybersécurité

Document modifiable – à compléter avec le nom, la classe et la date

1. Présentation du projet

Le projet consiste à développer une API HTTP permettant de gérer une liste d'étudiants. L'API est réalisée avec Flask et communique avec une base de données MySQL. Les principales opérations correspondent aux opérations CRUD : consulter, créer, modifier et supprimer des étudiants.

Le code fourni montre également une évolution progressive de l'API : les premières versions mettent en place les routes de gestion des étudiants, puis l'authentification est ajoutée. Une version plus récente sépare l'accès à la base de données dans une classe Database.

2. Description du projet – diagramme de cas d'utilisation

Le diagramme ci-dessous présente de manière neutre les interactions entre l'administrateur et l'API. L'administrateur peut s'identifier puis accéder aux fonctions de gestion des étudiants.

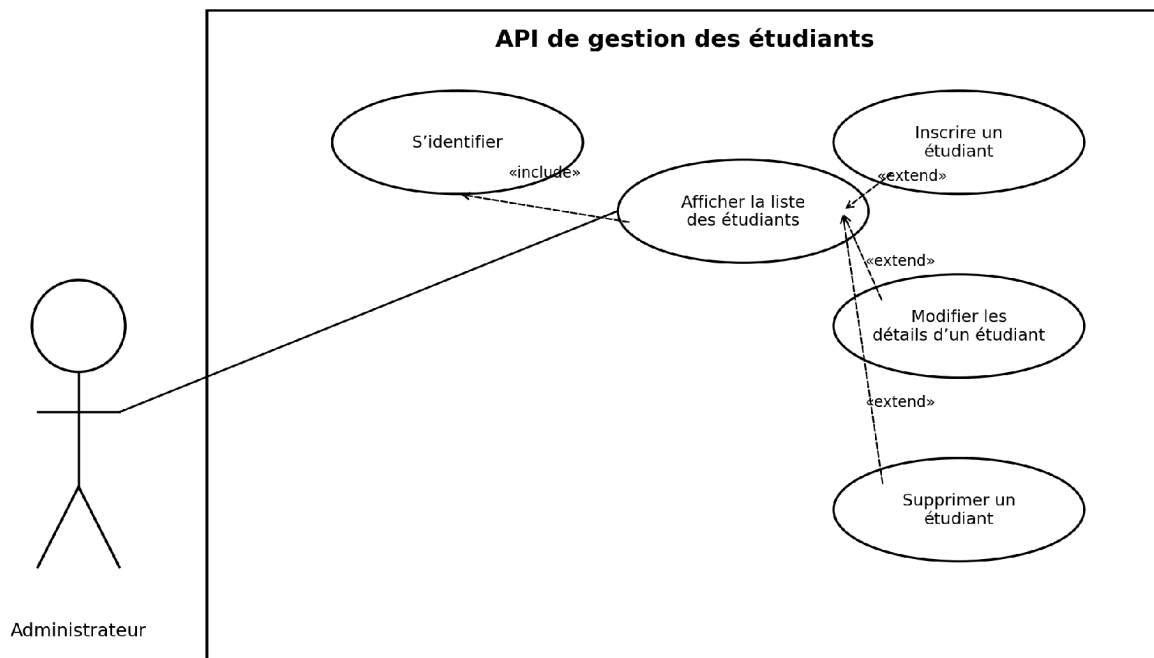


Figure 1 – Diagramme de cas d'utilisation de l'API.

2.1 Acteur

Administrateur : acteur qui utilise l'API pour consulter et gérer les informations relatives aux étudiants.

2.2 Cas d'utilisation

- S'identifier : vérifier les identifiants transmis à l'API.
- Afficher la liste des étudiants : récupérer les étudiants présents dans la base.
- Inscrire un étudiant : créer un nouvel enregistrement.
- Modifier les détails d'un étudiant : mettre à jour les informations d'un étudiant existant.
- Supprimer un étudiant : supprimer un étudiant identifié par son identifiant.

3. Description de l'API

L'API utilise HTTP pour permettre au client d'échanger avec le serveur. Les méthodes utilisées dans le projet sont GET, POST, PUT et DELETE. Les réponses des routes testées sont principalement des données JSON et des codes HTTP indiquant le résultat de l'opération.

3.1 Requêtes HTTP

Opération	Méthode	Route V4 testée	But	Réponse observée
Liste des étudiants	GET	/v4/etudiants/	Récupérer la liste	200 OK
Ajouter un étudiant	POST	/v4/etudiants/	Créer un étudiant	201 Created
Récupérer par ID	GET	/v4/etudiants/11	Récupérer un étudiant	200 OK
Modifier un étudiant	PUT	/v4/etudiants/1	Modifier un étudiant	200 OK
Supprimer un étudiant	DELETE	/v4/etudiants/10	Supprimer un étudiant	200 OK

Remarque : les routes ci-dessus sont celles visibles dans la capture Postman fournie. Les détails exacts du corps JSON des requêtes POST et PUT ne sont pas visibles dans la capture et ne sont donc pas reproduits ici.

3.2 Documentation avec Swagger / OpenAPI

Swagger peut être utilisé pour documenter les endpoints de l'API : méthode HTTP, URL, paramètres, corps de requête, réponses et mécanismes d'authentification. Dans le rapport final, une capture de l'interface Swagger peut être ajoutée à cette section si elle est disponible.

Exemple de présentation d'un endpoint :

GET /v4/etudiants/

Objectif : récupérer la liste des étudiants.

Réponse attendue : HTTP 200 et tableau JSON.

4. Récapitulatif des versions de l'API

Le dépôt contient plusieurs versions de l'API. La consigne demande un récapitulatif de trois versions ; les trois premières sont présentées ci-dessous. La V4 est ensuite mentionnée séparément car elle correspond à la version utilisée dans la capture Postman fournie.

Version	Évolution	Fonctionnement	Observation
---------	-----------	----------------	-------------

V1	Mise en place des opérations de gestion des étudiants.	Routes CRUD.	Pas d'authentification dans la première version.
V2	Ajout de l'authentification HTTP Basic et gestion d'un étudiant non trouvé.	CRUD + authentification.	Les contrôles d'accès sont introduits.
V3	Nouvelle version des routes avec authentification.	CRUD + authentification sur les routes concernées.	Évolution de l'API par rapport aux versions précédentes.

4.1 Version V4 – évolution supplémentaire

Le dépôt contient également une V4. Cette version introduit une classe Database destinée à regrouper les opérations d'accès à MySQL. La capture Postman fournie montre que les tests ont été réalisés sur les routes /v4/etudiants/.

5. Diagramme de classes – classe Database

La classe Database centralise la connexion à la base et les opérations de lecture, création, modification, suppression et authentification. Elle possède les attributs host, user, password et database.

Élément	Type	Rôle
host	Attribut	Adresse du serveur MySQL
user	Attribut	Utilisateur MySQL
password	Attribut	Mot de passe MySQL
database	Attribut	Nom de la base de données
connect()	Méthode	Créer une connexion MySQL
readAll()	Méthode	Récupérer tous les étudiants
readOne(id)	Méthode	Récupérer un étudiant
create(...)	Méthode	Créer un étudiant
update(...)	Méthode	Modifier un étudiant
delete(id)	Méthode	Supprimer un étudiant
login(request)	Méthode	Vérifier les identifiants

Le fichier db.py confirme la présence de ces attributs et méthodes, avec une méthode connect() qui utilise mysql.connector et des méthodes CRUD pour la table etudiant.

6. Fiche des tests de l'API réalisés avec Postman

La capture fournie correspond à une exécution de la collection Postman. Le récapitulatif affiché indique : 9 tests réussis, 0 test échoué, 0 test ignoré, 0 erreur et un temps moyen de réponse de 22 ms.

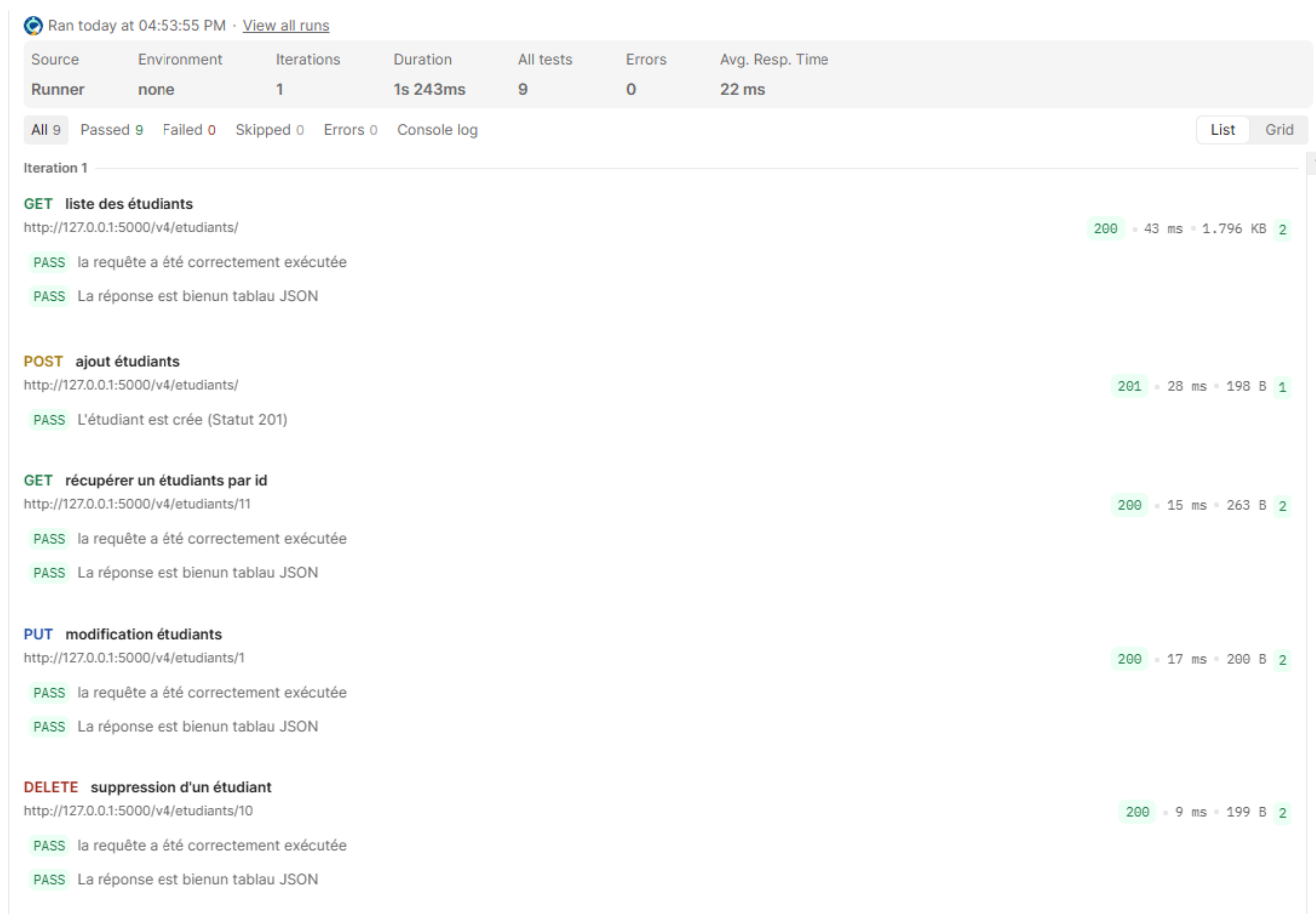


Figure 2 – Résultats de l'exécution de la collection Postman.

N°	Test	Méthode	Route	Code	Résultat
1	Liste des étudiants	GET	/v4/etudiants/	200	Réussi
2	Ajout étudiants	POST	/v4/etudiants/	201	Réussi
3	Récupérer un étudiant par ID	GET	/v4/etudiants/11	200	Réussi
4	Modification étudiants	PUT	/v4/etudiants/1	200	Réussi
5	Suppression d'un étudiant	DELETE	/v4/etudiants/10	200	Réussi

La capture ne montre que cinq requêtes, alors que le résumé Postman indique neuf tests. Les quatre assertions supplémentaires ne sont pas détaillées ici afin de ne pas inventer d'informations non visibles.

7. Conclusion et améliorations possibles en cybersécurité

Le projet permet de mettre en œuvre une API de gestion d'étudiants reliée à une base MySQL. Les tests Postman fournis montrent que les opérations principales de la V4 ont été exécutées avec succès.

L'analyse du fichier db.py met toutefois en évidence plusieurs points qui pourraient être améliorés avant une utilisation en environnement réel.

Requêtes SQL paramétrées : Les requêtes sont actuellement construites avec des f-strings. Il est recommandé d'utiliser des requêtes paramétrées afin de réduire le risque d'injection SQL.

Protection des mots de passe : La méthode login() compare directement le mot de passe reçu avec une valeur stockée en base. Une solution plus sûre consiste à stocker un hachage sécurisé du mot de passe et à utiliser une vérification adaptée.

HTTPS : Les échanges entre le client et l'API devraient être protégés par HTTPS afin de chiffrer les identifiants et les données transportées.

Validation des entrées : Les données reçues pour nom, prénom, e-mail, téléphone et identifiant devraient être contrôlées avant leur utilisation.

Gestion des erreurs : Les blocs except: très généraux masquent les causes précises des erreurs. Une gestion plus précise des exceptions permettrait de journaliser les erreurs sans exposer d'informations internes au client.

Authentification et autorisations : L'authentification peut être complétée par une gestion des droits afin de limiter les opérations selon le rôle de l'utilisateur.

Mode debug : Le mode debug doit être désactivé dans un environnement de production afin d'éviter l'exposition d'informations techniques.

Secrets de connexion : Les informations de connexion à la base doivent être conservées hors du code source, par exemple dans des variables d'environnement ou un gestionnaire de secrets.

8. Conclusion générale

Ce projet a permis de développer une API Flask dédiée à la gestion d'étudiants et d'en faire évoluer progressivement les fonctionnalités. Les différentes versions montrent l'ajout de mécanismes d'authentification et la séparation progressive de l'accès à la base de données grâce à la classe Database. Les tests Postman fournis montrent que les principales opérations de la V4 ont été exécutées avec succès.

Les prochaines améliorations peuvent principalement porter sur la sécurité : requêtes SQL paramétrées, protection des mots de passe, HTTPS, validation des entrées, contrôle des autorisations, gestion précise des erreurs et protection des informations de connexion. Ces améliorations renforceraient la robustesse de l'API et sa capacité à être utilisée dans un environnement réel.

Annexe – éléments à compléter avant remise

- Ajouter le nom, prénom, classe et date.
- Ajouter une capture de Swagger si elle est disponible.
- Vérifier les chemins exacts des endpoints dans les fichiers V1/V2/V3 avant impression.
- Si l'enseignant demande explicitement un diagramme Pyreverse, joindre la sortie Pyreverse officielle du fichier db.py.